

## Lecture 3: Logistic Regression

*Instructor: Jonathan Pipping-Gamón**Author: RB, JP*

## 3.1 Example: Sinking Putts

### 3.1.1 Problem Setup

**Question:** Predict the **probability** that a putt is sunk as a function of the distance to the hole.

**Data:** 266,846 first putts from PGA Tour ShotLink data, including distance to the hole and whether the putt was sunk or not. Each row in the dataset is a single putt and includes the following variables:

- $i$ : index of the  $i^{th}$  putt in our dataset
- $y_i$ : 1 if the  $i^{th}$  putt was sunk, 0 otherwise
- $x_i$ : distance to the hole of the  $i^{th}$  putt, measured in feet

**Visualization:** The first thing we should do is plot the data, which we do in Figure 3.1. What do you notice?

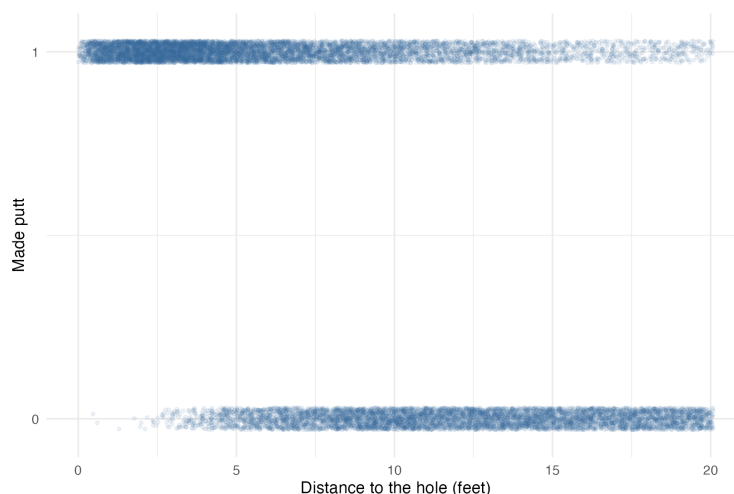


Figure 3.1: Scatter plot of the putt data

Like we would expect, the observed proportion of putts sunk decreases as the distance to the hole increases. This example will force us to separate two questions:

- What functional form gives us the right *shape*?
- What model form gives us a valid *probability*?

We begin by seeing how far ordinary linear regression can be pushed with transformed predictors. That improves the fit, but it does not solve the probability problem.

### 3.1.2 Modeling

#### Model 1: Simple Linear Regression

We know from Lecture 1 that this model takes the form:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \quad \mathbb{E}[\epsilon_i] = 0$$

We already know how to estimate the coefficients for this model. Recall that the least squares solution to this model is given by:

$$\begin{aligned} \hat{\beta}_0 &= \bar{Y} - \hat{\beta}_1 \bar{X} \\ \hat{\beta}_1 &= \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} \end{aligned}$$

We fit this model in R with the following code:

```
model_1 = lm(putt_made ~ distance, data = putt_data)  
summary(model_1)
```

When we plot the fitted line against the data in Figure 3.2, we see that the model does not fit the data well.

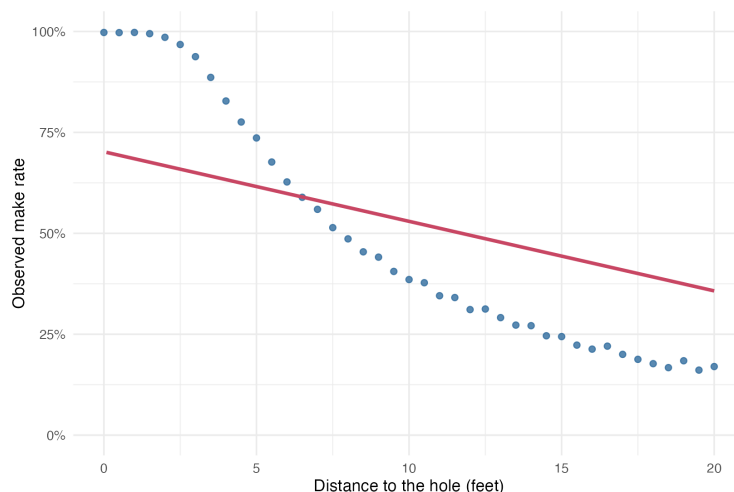


Figure 3.2: Putt data and fitted simple linear regression

The issue here is that the relationship is not close to linear in raw distance. The make rate drops very quickly for short putts, then flattens out for longer ones. So before giving up on linear regression entirely, it is reasonable to ask whether a transformed predictor would do better. A natural next step is to work with  $\log(x_i)$ .

#### Model 2: Linear Regression with a Log-Distance Transformation

This transformed model takes the form

$$y_i = \beta_0 + \beta_1 \log(x_i) + \epsilon_i, \quad \mathbb{E}[\epsilon_i] = 0$$

We estimate the coefficients in exactly the same way as before. The only thing that changes is the predictor we feed into the model. In matrix notation, the least-squares solution is still

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

We fit this model in R with the following code:

```
model_2 = lm(putt_made ~ log_distance, data = putt_data)  
summary(model_2)
```

When we plot the fitted line against the data in Figure 3.3, we see that the transformed model fits the short-putt region much better.

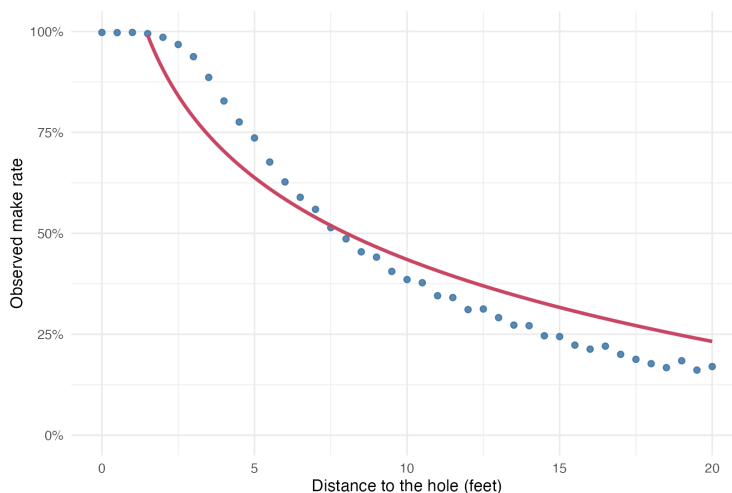


Figure 3.3: Putt data and fitted linear regression on log distance

However, when we zoom out in Figure 3.4, we still run into the same core problem: this is ordinary linear regression, so the fitted values are not constrained to stay in  $[0, 1]$ .

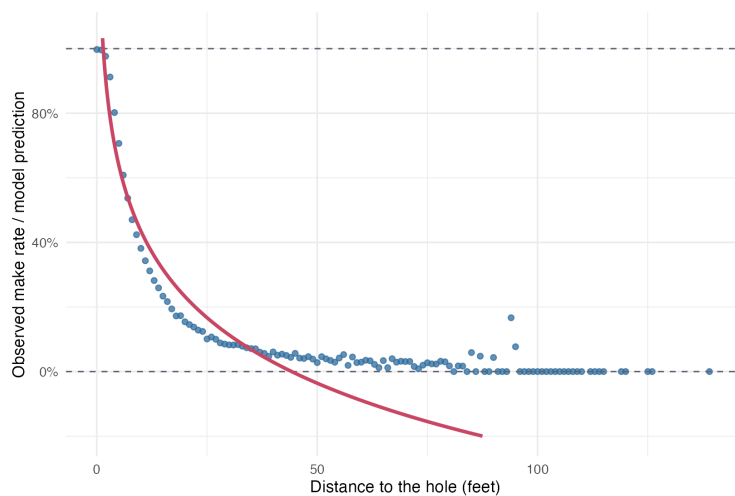


Figure 3.4: Putt data and fitted linear regression on log distance (zoomed out)

So the transformed linear model gets the *shape* more nearly right, but it is still the wrong modeling framework. We want to predict a probability, and probabilities must stay between 0 and 1. More fundamentally, if  $Y_i \sim \text{Bernoulli}(p_i)$ , then

$$\text{Var}(Y_i | x_i) = p_i(1 - p_i),$$

so binary outcomes do not naturally follow a constant-variance additive-error model. We need a model built for probabilities.

## 3.2 Logistic Regression

### 3.2.1 The Logistic Function

The next step is not just a more flexible line. We need a function that takes any real number and turns it into a valid probability. One such function is the **logistic (or sigmoid) function**:

**Definition 3.1** (Logistic Function). *The **logistic function** is defined as:*

$$\text{Logistic}(z) = \text{sigmoid}(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

We plot the logistic function in Figure 3.5.

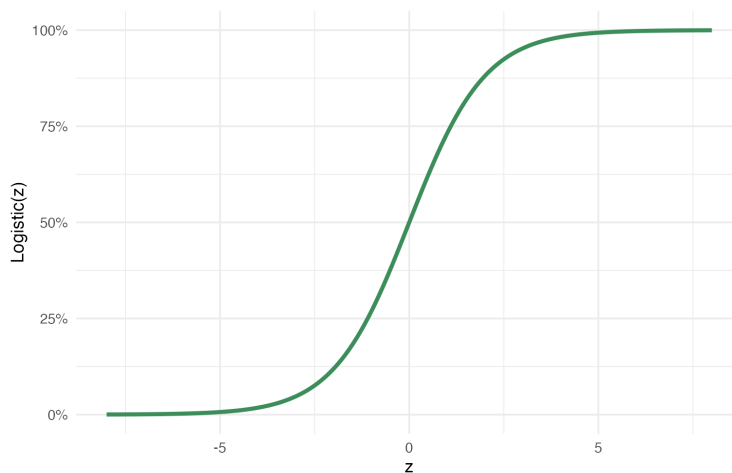


Figure 3.5: The Logistic Function

The idea is simple: first build a linear score, then pass that score through the logistic function to turn it into a probability.

### 3.2.2 The Model

As in linear regression, we start with a linear predictor:

$$\eta_i = \beta_0 + \beta_1 z_i$$

where  $z_i$  is whichever transformed predictor we want to use. Here, a good choice is  $z_i = \log(x_i)$ . The difference is that we no longer interpret  $\eta_i$  as the prediction itself. Instead,  $\eta_i$  is a score on the **log-odds scale**, and we convert it into a probability by applying the logistic function:

$$p_i = \text{Logistic}(\eta_i) = \text{Logistic}(\beta_0 + \beta_1 z_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 z_i)}}$$

This is a **logistic regression model**. The outcome  $y_i$  is still binary, but the model outputs the conditional probability that  $y_i = 1$ . Mathematically, we can write this model as:

$$p_i = \mathbb{P}(y_i = 1 \mid x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 z_i)}}$$

where  $z_i = \log(x_i)$  and  $y_i \sim \text{Bernoulli}(p_i) = \begin{cases} 1 & \text{w.p. } p_i \\ 0 & \text{w.p. } 1 - p_i \end{cases}$

Equivalently, because the logistic function is invertible,

$$\log\left(\frac{p_i}{1 - p_i}\right) = \beta_0 + \beta_1 z_i.$$

That is why logistic regression coefficients are interpreted on the **log-odds scale**.

### 3.2.3 Estimating the Coefficients

Our data gives us observed outcomes  $y_i \in \{0, 1\}$ , not the probabilities  $p_i$  themselves. In linear regression, we chose  $\beta$  by minimizing squared error. In logistic regression, we choose  $\beta$  by minimizing the negative log-likelihood, also called the **log loss**:

$$L(\beta) = -\frac{1}{n} \sum_{i=1}^n (y_i \log p_i + (1 - y_i) \log(1 - p_i))$$

$$\text{where } p_i = \frac{1}{1 + e^{-\mathbf{x}_i^T \beta}}$$

The intuition is straightforward:

- If  $y_i = 1$ , then observation  $i$  contributes  $-\log p_i$
- If  $y_i = 0$ , then observation  $i$  contributes  $-\log(1 - p_i)$

So confident correct predictions cost very little, while confident wrong predictions cost a lot. This objective is **convex** in  $\beta$ , but unlike least squares it has no closed-form solution. In practice, `glm(..., family = "binomial")` computes the maximum-likelihood estimate numerically via iteratively reweighted least squares.

## 3.3 Back to Sinking Putts

We now return to the putt example. The response is still  $y_i \in \{0, 1\}$ , and we will use

$$z_i = \log(x_i)$$

as the predictor, where  $x_i$  is distance in feet. Since every putt distance is positive, this transform is well-defined.

### 3.3.1 A First Logistic Model

We start by putting the log-distance predictor into a logistic regression:

```
model_3 = glm(putt_made ~ log_distance,
              data = putt_data, family = "binomial")
summary(model_3)
```

We plot the fitted line against the data in Figure 3.6. It seems to fit the data well, but we should check the model's performance from longer distances.

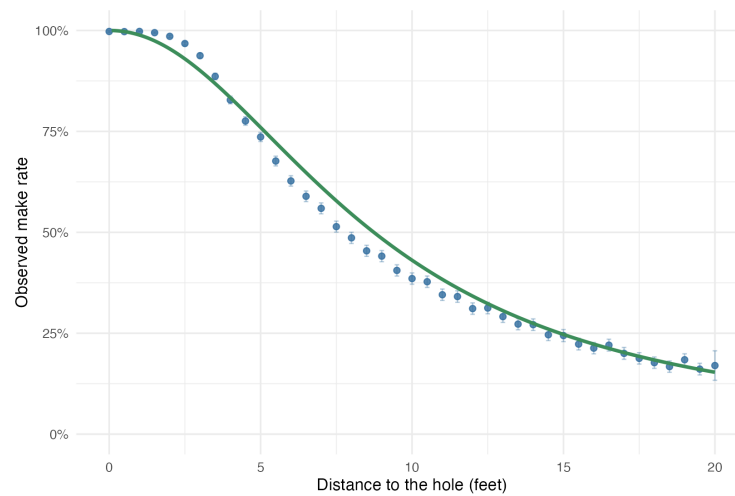


Figure 3.6: Putt data and fitted logistic regression

The fit now has the right overall shape, and the fitted values stay between 0 and 1 by construction. That fixes the main structural problem with linear regression. The remaining question is whether a straight line in log distance is flexible enough.

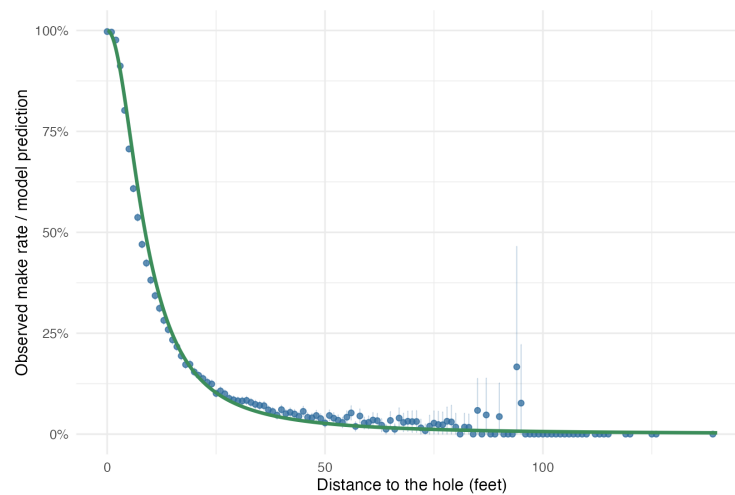


Figure 3.7: Putt data and fitted logistic regression (zoomed out)

### 3.3.2 Improving the Model

The simple log-distance logistic model is reasonable, but the observed make-rate curve still bends a bit more than a straight line on the log-odds scale. A natural next step is to let the log odds be cubic in log distance:

$$\mathbb{P}(y_i = 1 \mid x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \log(x_i) + \beta_2 [\log(x_i)]^2 + \beta_3 [\log(x_i)]^3)}}$$

We fit this model in R with the following code:

```

model_4 = glm(putt_made ~ log_distance + I(log_distance^2) +
              I(log_distance^3),
              data = putt_data, family = "binomial")
summary(model_4)

```

We plot the fitted line against the data in Figure 3.8, and we can see that it tracks the observed make-rate curve more closely while still respecting the  $[0, 1]$  constraint.

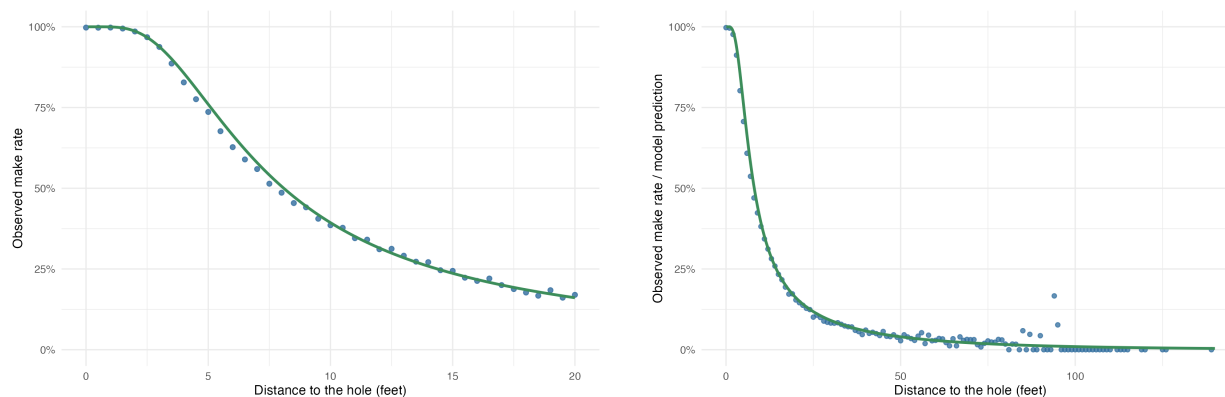


Figure 3.8: Putt data and fitted logistic regression with a cubic in log distance

### 3.3.3 Interpretation and Uncertainty

Once we have a model that fits reasonably well, we can interpret the output and quantify uncertainty. For a fitted logistic model, software reports coefficient estimates, standard errors, and confidence intervals on the log-odds scale. In R, the usual summaries are

```

summary(model_4)
confint(model_4)

```

Appendix Appendix A gives the matrix formulas behind these summaries.

Because logistic Regression coefficients live on the log-odds scale,  $e^{\beta_j}$  is an odds ratio. Exponentiating a confidence interval for  $\beta_j$  gives a confidence interval for the corresponding odds ratio.

If we want uncertainty for the fitted probability at a new predictor vector  $\mathbf{x}_0$ , the cleanest workflow is:

1. compute uncertainty on the linear-predictor, or log-odds, scale;
2. form a confidence interval there;
3. transform the interval endpoints through the logistic function.

Working on the link scale first is what keeps the final interval on the probability scale inside  $(0, 1)$ . Appendix Appendix A writes these steps out algebraically.

Finally, even if the underlying probability were known exactly, the outcome of one new putt would still be random:

$$Y_0 \mid x_0 \sim \text{Bernoulli}(p_0).$$

So the outcome variance depends on the probability itself,  $p_0(1 - p_0)$ ; it is largest near 0.5 and smallest near 0 or 1.

### 3.3.4 Takeaways

What can we take away from this example?

- Use linear regression when the response can take any real value.
- Use logistic regression when the response is binary and the quantity of interest is a **probability**.
- In this example, it helps to separate two tasks: build enough flexibility to capture the shape, then choose a model family that respects the response type.
- Logistic Regression coefficients describe changes in **log odds**; exponentiating them gives odds ratios.
- When transformed or polynomial terms are present, interpretation applies to the whole linear predictor. In the final model, the effect of log distance on the log odds is  $\beta_1 + 2\beta_2 \log(x_i) + 3\beta_3 [\log(x_i)]^2$ , not just  $\beta_1$ .

## Appendix A Formulas for Logistic Regression Uncertainty

This appendix records the matrix formulas behind the uncertainty discussion in the main lecture.

For a fitted logistic model,

$$\widehat{\boldsymbol{\beta}} \approx N\left(\boldsymbol{\beta}, (\mathbf{X}^T \widehat{\mathbf{W}} \mathbf{X})^{-1}\right), \quad \widehat{W}_{ii} = \widehat{p}_i(1 - \widehat{p}_i).$$

So the estimated variance-covariance matrix is

$$\widehat{\text{Var}}(\widehat{\boldsymbol{\beta}}) = (\mathbf{X}^T \widehat{\mathbf{W}} \mathbf{X})^{-1},$$

and the standard error of  $\widehat{\beta}_j$  is the square root of the  $j$ th diagonal entry.

If we want uncertainty for the fitted probability at a new predictor vector  $\mathbf{x}_0$ , it is simplest to work on the linear-predictor scale first:

$$\widehat{\eta}_0 = \mathbf{x}_0^T \widehat{\boldsymbol{\beta}}, \quad SE(\widehat{\eta}_0) = \sqrt{\mathbf{x}_0^T \widehat{\text{Var}}(\widehat{\boldsymbol{\beta}}) \mathbf{x}_0}.$$

An approximate 95% confidence interval for the log odds is

$$\widehat{\eta}_0 \pm 1.96 SE(\widehat{\eta}_0).$$

Transforming those endpoints through the logistic function gives an approximate 95% confidence interval for the fitted probability:

$$\left[ \frac{1}{1 + e^{-(\widehat{\eta}_0 - 1.96 SE(\widehat{\eta}_0))}}, \frac{1}{1 + e^{-(\widehat{\eta}_0 + 1.96 SE(\widehat{\eta}_0))}} \right].$$